

R Lab6. Estructura de proteína con R

Bioinformática. Ingeniería Biomédica

Las proteínas, a diferencia de los nucleótidos, muestran una gran variedad de conformaciones tridimensionales necesarias para sus diversos roles estructurales y funcionales. Los aminoácidos en una proteína van a determinar su estructura tridimensional, por lo que la frecuencia de algunos aminoácidos o combinaciones de ellos en la secuencia juega un papel importante en la determinación de las estructuras secundarias de la proteína: hélices α , láminas β o hélices de colágeno.

Una tarea importante será predecir la estructura secundaria de un péptido, que algunas veces es un poco más complicado que evaluar la frecuencia y posición de determinados aminoácidos.

Por otra parte, ciertas propiedades físicoquímicas de una proteína también están determinadas por las propiedades de los aminoácidos que conforman su secuencia. Por ejemplo, la acidez se puede decidir en función de la fracción de aminoácidos ácidos en la proteína.

Todas las propiedades de las proteínas, desde la estructura hasta las físicoquímicas, son fundamentales para determinar su función biológica.

En este laboratorio vamos a trabajar en R con proteínas, en sus niveles de secuencia y estructura.

El paquete Bio3D

En realidad el paquete Bio3D es un conjunto de paquetes de R que contiene utilidades para el análisis de datos de estructura, secuencia y trayectoria biomolecular, aprovechando las capacidades gráficas y estadísticas del entorno R y, siendo muy útil para el análisis exploratorio de datos estructurales.

La funcionalidad para varias tareas de análisis complejas está empaquetada en paquetes separados, que incluyen:

- **bio3d** que es el paquete básico, que incluye alineación, comparaciones de secuencia y estructura, y análisis inter-conformacional con PCA.
- **bio3d.nma** dirigido a predecir y contrastar dinámicas funcionales entre familias de proteínas.
- **bio3d.cna** para análisis de estructura de proteínas y redes de correlación para caracterizar movimientos proteicos correlacionados subyacentes a la regulación alostérica.
- **bio3d.web** para realizar análisis interactivos en línea de estructuras de proteínas y su dinámica.
- **bio3d.eddm** para caracterizar cambios conformacionales significativos funcionalmente.
- **bio3d.view** para visualización interactiva en 3D.

En este laboratorio no es posible explorar todas las opciones, por lo que haremos un repaso por las cuestiones más básicas.

Instala y carga el paquete **bio3d** disponible en CRAN con todas sus dependencias (argumento **dependencies = TRUE** de **install.packages()**)

Trabajar con ficheros PDB

Los archivos del Banco de Datos de Proteínas (PDB, *Protein Data Bank*) son el formato más común para la distribución y almacenamiento de datos de coordenadas biomoleculares de alta resolución. En su forma más básica, los archivos de coordenadas PDB contienen una lista de todos los átomos de una o más estructuras moleculares. La posición de cada átomo está definida por sus coordenadas x, y, z en un sistema de coordenadas ortogonal convencional. Además, los archivos PDB comúnmente (pero no siempre) incluyen datos adicionales, como listas de elementos de estructura secundaria observados. El paquete `bio3d` contiene funciones para la lectura, escritura y manipulación de archivos PDB.

Leer un fichero PDB

1. Utiliza la función `read.pdb()` para leer la información sobre el ID '4q21' (<https://www.rcsb.org/structure/4q21>). Esta función requiere como argumento la especificación del archivo que se va a leer, que puede ser tanto el nombre de un archivo local o el identificador PDB de un archivo del repositorio PDB. ¿De qué clase es el objeto obtenido?

```
## Note: Accessing on-line PDB file
```

2. La última parte de la información del objeto `pdb` son los atributos (`+attr`). Es posible obtener los atributos de un objeto utilizando la función `attributes()`,

```
attributes(pdb)
```

```
## $names
## [1] "atom" "xyz" "seqres" "helix" "sheet" "calpha" "remark" "call"
##
## $class
## [1] "pdb" "sse"
```

Para acceder a cada uno de estos atributos de forma individual, utilizamos la sintaxis común de las listas en R. Obtén la información del atributo átomo o componente.

Seleccionar átomos

En muchas ocasiones nos interesa seleccionar subconjuntos de átomos que tengan alguna característica de interés. Por ejemplo,

```
(ca.indxs <- atom.select(pdb, "calpha"))
```

```
##
## Call: atom.select.pdb(pdb = pdb, string = "calpha")
##
## Atom Indices#: 168 ($atom)
## XYZ Indices#: 504 ($xyz)
##
## + attr: atom, xyz, call
```

Esta instrucción selecciona los átomos de carbono α (C-alpha) de la proteína. El C-alpha es el átomo central de cada aminoácido y sirve como referencia estructural porque está presente en todos los residuos y define el esqueleto de la proteína.

La función `atom.select()` devuelve un objeto que contiene los identificadores de los átomos seleccionados (`$atom`) y sus coordenadas tridimensionales (`$xyz`). Esto permite extraer fácilmente las posiciones o propiedades de esos átomos para análisis posteriores. Se pueden consultar otras opciones disponibles con `?atom.select`.

3. El argumento `inverse = TRUE` de la función `atom.select()` permite invertir la selección. Selecciona todos los átomos excepto los átomos de agua (`water`).

4. El argumento `operator` de `atom.select()` permite combinar varios criterios de selección. Podemos usar `operator="AND"` para que se cumplan todos los criterios a la vez, o `operator="OR"` para que se cumpla al menos uno de ellos. Selecciona todos los átomos que pertenezcan a la proteína (`string = "protein"`) o bien que pertenezcan a un ligando llamado GDP (`resid = "GDP"`).

5. Repite la selección utilizando la función `combine.select()`, que permite combinar dos objetos de selección (por ejemplo, dos resultados de `atom.select()`) mediante operadores lógicos como `AND`, `OR`, o `NOT` (argumento `operator`).

Trabajar con múltiples ficheros PDB

La función `cat.pdb()` se puede utilizar para concatenar dos o más archivos PDB. Esta función contiene argumentos adicionales para reasignar números de residuos e identificadores de cadena. En el ejemplo a continuación, ilustramos cómo concatenar '4q21' con componentes específicos de '4lhy' (<https://www.rcsb.org/structure/4LHY>) en un nuevo objeto PDB,

```
# leer los dos pdb
a <- read.pdb("4q21")
b <- read.pdb("4lhy")

## trim.pdb() crea un nuevo objeto PDB basado en una seleccion
# cadena A de 4q21
a1 <- trim.pdb(a, chain="A")

# cadenas A, E y F de 4lhy
b1 <- trim.pdb(b, chain="A")
b2 <- trim.pdb(b, chain="E")
b3 <- trim.pdb(b, chain="F")

# creamos un nuevo objeto pdb uniendo esas estructuras
new <- cat.pdb(a1, b1, b2, b3, rechain=TRUE)
# rechain para asignar nuevos identificadores de cadena
unique(new$atom$chain)

# write.pdb() para escribir un fichero PDB
write.pdb(new, file="4Q21-4LHY.pdb")
```

Visualizaciones 3D de proteínas

Cuando estamos investigando una proteína es importante poder visualizarla en tres dimensiones para tareas como el diseño de fármacos o el modelado de proteínas. En R podemos visualizar estas estructuras en

gráficos interactivos utilizando el paquete `NGLViewer` disponible en CRAN. La información que se maneja es la contenida en la base de datos PDB.

0. Instala y carga la librería.

Vamos a ilustrar el uso del paquete `NGLViewer` utilizando como ejemplo el dominio motor de la quinesina humana, cuyo identificador en el PDB es '1BG2'.

1. Para cargar la información de una proteína y visualizarla en 3D, utilizamos la función `NGLViewer()`, indicando como argumento el ID PDB de la proteína (en este caso, "1BG2").

Añadiendo `addRepresentation("cartoon")`, obtenemos una representación esquemática básica de la estructura.

Podemos encadenar ambas instrucciones usando el operador `%>%` para hacerlo de forma más compacta:

```
NGLViewer("1BG2") %>%  
addRepresentation("cartoon")
```

Esta instrucción (1) Carga la proteína "1BG2" desde PDB; y (2) la representa en forma de "cartoon", una vista simplificada que muestra la estructura secundaria (hélices, láminas, etc.).

El gráfico que se genera es completamente interactivo. Podemos girar y mover la proteína con el botón izquierdo del ratón, alejar o acercar usando la rueda, y seleccionar regiones de la estructura con un clic.

2. A la función `NGLViewer()` le pasamos como argumento el identificador PDB de una proteína o un fichero en formato pdb. Ahora, representa la estructura que acabamos de crear y guardar previamente en el archivo local `4Q21-4LHY.pdb`.

Tipos de representación y combinaciones

La estructura 3D puede ser representada como "cartoon", "ball+stick", "line", "surface" o "ribbon". Podéis consultar las representaciones disponibles en el manual `NGL.js` en el epígrafe *Molecular Representations*.

3. Es posible utilizar varios tipos de representación en un mismo gráfico, utilizando la función `addRepresentation()` tantas veces como queráis. Representa la proteína 1BG2 superponiendo dos representaciones distintas.

4. También se le puede añadir una animación con las funciones `setRock()` o `setSpin()`. ¿Qué diferencia hay entre estas dos opciones?

Personalización visual por colores y regiones

5. Otro argumento que se puede utilizar de la función `NGLViewer::addRepresentation()` es `param`. Con este parámetro podemos cambiar el aspecto de algunos residuos. Debemos pasarle una lista especificando las características que nos interese modificar.

5.1. Utiliza `colorScheme = "residueindex"` para colorear la proteína según el índice de residuo.

Podemos seleccionar y representar regiones concretas. Por ejemplo para representar los residuos entre las posiciones 233-248 podemos utilizar el elemento `sele` de la lista `param`. Añadiendo elementos a la lista podemos modificar distintos aspectos, por ejemplo el color,

```
# seleccionamos un grupo de AA con sele  
NGLViewer("1BG2") %>%  
  addRepresentation("ball+stick",  
    param = list(sele = "233-248",  
                  colorValue = "yellow",  
                  colorScheme = "element"))
```

5.2. Representa la proteína completa en forma de “cartoon” y superpón este segmento en forma de “ball+stick”.

Etiquetado de residuos

Es posible etiquetar aminoácidos utilizando la función `addRepresentation()` con el parámetro `label`. Utilizamos `sele` para seleccionar un aminoácido con la segunda llamada `addRepresentation()`. La tercera llamada sirve para etiquetar ese aminoácido.

```
NGLViewer("1BG2") %>%
  addRepresentation("cartoon") %>%
  addRepresentation("ball+stick",
    param = list(colorScheme = "element",
                  colorValue = "yellow",
                  sele = "20")) %>%
  addRepresentation("label", param = list(sele = "20",
    labelType = "format",
    labelFormat = "[% (resname)s] % (resno)s",
    labelGrouping = "residue",
    color = "white", fontFamily = "sans-serif",
    xOffset = 1, yOffset = 0, zOffset = 0,
    fixedSize = TRUE, radiusType = 1, radiusSize = 1.5))
```

Esquematizar la información disponible en *UniProt* de una proteína

Otra base de datos sobre proteínas que solemos manejar es *UniProt*. Esta base de datos proporciona información detallada sobre secuencias de proteínas, anotaciones funcionales, estructuras, expresión, interacciones, etc, centrandose en la secuencia y función de las proteínas más que en la estructura tridimensional como *PDB*. Desde R podemos utilizar la información disponible en *UniProt* para crear esquemas sobre una proteína utilizando el paquete `drawProteins` de Biconductor.

0. Instala y carga el paquete.

1. Utiliza la función `get_features()` para recuperar información estructural y funcional de la proteína: secuencia, dominios, regiones desordenadas, motivos, sitios de fosforilación, etc.

Almacena en un objeto la información obtenida con `get_features()` para la proteína Rel A o NF-kappaB, p65 que es un factor de transcripción de DNA muy conocido y cuyo identificador es “Q04206”. Podéis ver la información disponible en *UniProt* en el enlace (<https://www.uniprot.org/uniprot/Q04206>). ¿Que tipo de objeto se obtiene como resultado?

```
## [1] "Download has worked"
```

2. Para trabajar más fácilmente con los datos, convierte el objeto anterior en un `data.frame` con `feature_to_dataframe()`.

3. Vamos a esquematizar la información de esta proteína utilizando el paquete `ggplot2`.

Primero dibujamos el área proporcional a la longitud de la proteína usando `draw_canvas()`. Utiliza esta función con argumento el `data.frame` que has creado en el paso anterior. Almacena el resultado en un objeto `ggplot` y muestra el gráfico resultante.

3.1. Dibuja la cadena principal de la proteína con `draw_chains()`. Esta función requiere como primer argumento el objeto `ggplot` creado en el paso anterior y el `data.frame` con la información.

3.2. Añade los dominios estructurales con `draw_domains()`.

Podemos personalizar este gráfico utilizando las utilidades de `ggplot2`. Por ejemplo, si queremos eliminar los ejes, el fondo y controlar el tamaño del texto,

```
library("ggplot2")
p <- p + theme_bw(base_size = 12) + # white background
  theme(panel.grid.minor=element_blank(),
        panel.grid.major=element_blank()) +
  theme(axis.ticks = element_blank(),
        axis.text.y = element_blank()) +
  theme(panel.border = element_blank())
p
```

Añadir otras características

4. Hemos creado la estructura básica, pero podemos añadir muchas otras características de interés.

4.1. Regiones de interés con `draw_regions()`.

4.2. Motivos con `draw_motif()`.

4.3. Los sitios de fosforilación: regiones específicas en las proteínas donde los grupos fosfato (PO_4^{-2}) son agregados covalentemente a los aminoácidos en una modificación post-traducciona. Utiliza la función `draw_phospho()`.

4.4 Elementos de plegamiento comunes que contribuyen a su conformación tridimensional, como hélices alfa, láminas beta y giros, con `draw_folding()`.

4.5. Otras funciones que se pueden utilizar,

- Repeticiones con `draw_repeat(p, rela.data)`.
- La topología transmembrana con `draw_recept_dom()`.
- También podemos añadir títulos y leyendas utilizando la sintaxis de `ggplot2`, por ejemplo,

```
rela.subtitle <- paste0("circles = phosphorylation sites\n",
                        "RHD = Rel Homology Domain\n",
                        "source:Uniprot")
p <- draw_regions(p, rela.data)
p <- draw_phospho(p, rela.data, size = 8)
p <- p + labs(title = "Rel A/p65",
             subtitle = rela.subtitle)
p
```

Comparar varias proteínas

Estos esquemas pueden ser muy útiles para comparar la información de varias proteínas, especialmente si somos capaces de esquematizar todas ellas en el mismo gráfico.

5. Obtén la información de cinco proteínas de la familia NFkappaB con identificadores “Q04206”, “Q01201”, “Q04864”, “P19838” y “Q00653”. Utiliza la función `get_features()` con argumento un *string* con todos los identificadores separados por espacios.

6. Construye el gráfico,

```
## [1] "Download has worked"
```

Schematic of human NF-kappaB proteins

circles = phosphorylation sites

RHD = Rel Homology Domain

source:Uniprot

